Memoria técnica: arquitectura de seguridad y control de acceso

Proyecto: GastoFlow, gestión de gastos corporativos (Ionic 9 + Angular 22 standalone)
Módulo: Desarrollo de Interfaces (0488) · DAM 2 · Nebrija
Autor: Jorge González Gómez · **Rama de trabajo:** `develop`

> **Estado de implementación.** La memoria describe lo que ya está en el código (marcado **[Hecho]**) y lo que está diseñado y se implementa al conectar Supabase en la Fase 3 (marcado **[Fase 3]**). Los fragmentos de código son los del repositorio.

A. Fundamentación teórico-técnica

A.1 Functional Guards en Angular / Ionic standalone

Un **guard** es una función que el router ejecuta durante una navegación para decidir si se permite. Desde Angular 15 se escriben como **funciones** (`CanMatchFn`, `CanActivateFn`...) en lugar de clases que implementan una interfaz (`implements CanActivate`, ya obsoleto). Obtienen sus dependencias con `inject()` y devuelven:

- `true`: la navegación continúa.
- `false`: se cancela (en `canMatch`, la ruta se ignora y el router prueba la siguiente).
- `UrlTree`: se cancela y se **redirige** a esa URL (p. ej. `/login?returnUrl=...`).
- También pueden devolver `Promise` u `Observable` de cualquiera de los anteriores.

Ciclo de vida dentro de una navegación:

1. `NavigationStart`.
2. **Reconocimiento de la URL**: el router recorre el árbol de rutas. Aquí se ejecuta **`canMatch`**, *antes* de dar una ruta por coincidente (y antes de descargar su `loadChildren`).
3. **Comprobación de guards** sobre las rutas ya elegidas: primero **`canDeactivate`** de la página que se abandona y después **`canActivateChild`** y **`canActivate`** de las nuevas.
4. Resolvers (`resolve`).
5. Descarga de los componentes con `loadComponent` y activación de la vista (`NavigationEnd`).

Tipo	Cuándo se ejecuta	Para qué sirve
`canMatch`	Al reconocer la URL	Decidir si la ruta "existe" para este usuario. Evita descargar código ajeno y permite tener la misma URL con componentes distintos según el rol. Sustituye al antiguo `canLoad`.
`canActivate`	Con la ruta ya reconocida	Permitir o no entrar en una ruta concreta.
`canActivateChild`	Antes de entrar en cualquier hija	Proteger un grupo de rutas hijas a la vez.
`canDeactivate`	Al intentar salir de una página	Evitar perder cambios sin guardar (p. ej. un formulario de gasto a medias).

`canActivate` frente a `canMatch` (rendimiento y lazy loading). Con `canActivate`, el router primero reconoce la ruta. Si usa `loadChildren`, la configuración hija **ya se ha descargado** para poder reconocerla, aunque luego el guard la bloquee. Con `canMatch` la decisión se toma antes: si falla, nada de esa rama se descarga ni se construye. Por eso GastoFlow protege la zona privada y las rutas de administración con `canMatch`.

Además se ha **quitado la precarga** (`PreloadAllModules`) que trae la plantilla de Ionic: precargaría en segundo plano todas las páginas, incluidas las de administración, aunque el usuario sea un empleado.

A.2 HTTP Interceptors (`HttpInterceptorFn`)

Un interceptor es una función por la que pasa **cada petición** de `HttpClient` antes de salir, y su respuesta al volver. Los interceptores forman una **cadena (pipeline)**: cada uno recibe la petición y la función `next` para pasarla al siguiente; el último la envía al servidor. Es el patrón *Chain of Responsibility*, el mismo que los `Filter` de Servlets en Java.

- **Cabecera de autorización.** Las peticiones (`HttpRequest`) son **inmutables**, así que el interceptor crea una copia con `req.clone({ setHeaders: { Authorization: 'Bearer <token>' } })`. Así ningún servicio tiene que acordarse de añadir la cabecera. Solo se añade a peticiones hacia la API propia, para no filtrar el token a terceros.

- **Errores de sesión centralizados.** Con `catchError` en la respuesta:
- **401 Unauthorized** (el token no vale o ha caducado): se intenta renovar la sesión una única vez. Si se renueva, se repite la petición original con el token nuevo. Si no, se cierra la sesión y se redirige a `/login?returnUrl=<página actual>`.
- **403 Forbidden** (sesión válida pero sin permiso): no se cierra la sesión. Se avisa al usuario y se le devuelve a su inicio.
- **Refresh token.** El *access token* dura poco (en Supabase, 1 hora) y el *refresh token* sirve para pedir uno nuevo sin volver a escribir la contraseña. Dos reglas: no intentar renovar desde la propia petición de renovación (evita bucles) y, si varias peticiones reciben 401 a la vez, que todas esperen a **una sola** renovación compartida.

A.3 Control de acceso basado en roles (RBAC)

RBAC (*Role-Based Access Control*, modelo NIST) define tres conjuntos y dos relaciones: **usuarios ↔ roles ↔ permisos**. Al usuario no se le dan permisos directamente: se le asigna un rol, y el rol agrupa permisos. Frente a modelar identidades sueltas ("el usuario 7 puede aprobar"), cambiar lo que puede hacer un tipo de usuario es editar **un** rol, no a cada persona.

	Autenticación (AuthN)	Autorización (AuthZ)
Pregunta	¿Quién eres?	¿Qué puedes hacer?
Cómo	Credenciales → sesión / token	Rol y permisos del token frente a la acción pedida
En GastoFlow	`AuthService.login()` y `authGuard`	`PERMISOS_POR_ROL`, `permisoGuard` y menú filtrado
Fallo HTTP	401	403

Encapsulamiento del token. Un JWT lleva en su *payload* unos **claims**: `sub` (id), `email`, `role` y `exp` (caducidad). GastoFlow guarda esos datos en un tipo `SesionClaims`, que tiene la misma forma que el payload. Solo los gestiona un **servicio central** (`AuthService`), que los expone como signals de solo lectura: `sesion`, `rol`, `permisos`, `estaAutenticado`. Guards, menú y páginas **preguntan al servicio**; ninguno lee el storage ni decodifica el token por su cuenta. Así, cambiar la sesión simulada por Supabase solo toca ese servicio.

B. Justificación de implementación en el proyecto

B.1 Ubicación en el árbol de rutas y servicios

Roles y pantallas (`src/app/core/models/rol.model.ts`) **[Hecho]**

Pantalla	Ruta	Permiso	`empleado`	`admin`
Login	`/login`	sin sesión (`guestGuard`)	—	—
Inicio, Ayuda	`/inicio`, `/ayuda`	sesión (`authGuard`)	Sí	Sí
Mis gastos	`/gastos`	`gastos:ver-propios`	Sí	Sí
Aprobaciones	`/admin/aprobaciones`	`gastos:aprobar`	**Bloqueada**	Sí
Cuadro de mando	`/admin/dashboard`	`dashboard:ver`	**Bloqueada**	Sí

```
// src/app/core/models/rol.model.ts  [Hecho]
export const PERMISOS_POR_ROL: Readonly<Record<Rol, readonly Permiso[]>> = {
  empleado: ['gastos:ver-propios', 'gastos:crear'],
  admin: ['gastos:ver-propios', 'gastos:crear', 'gastos:ver-todos', 'gastos:aprobar',
          'dashboard:ver', 'informes:exportar', 'usuarios:gestionar'],
};
```

Guards en `app.routes.ts` [Hecho]. Hay un `authGuard` en el padre de toda la zona privada y un `permisoGuard(...)` en cada ruta sensible. `permisoGuard` es una **factoría**: recibe el permiso y devuelve el guard configurado.

```
// src/app/app.routes.ts (extracto)
{ path: 'login', canMatch: [guestGuard], loadComponent: () => import('./features/auth/login/login.page')
... },
{
  path: '',
  canMatch: [authGuard],                       // ¿hay sesión válida?
  loadComponent: () => import('./layout/shell/shell.component') ...,
  children: [
    { path: 'gastos',             canMatch: [permisoGuard('gastos:ver-propios')], ... },
    { path: 'admin/aprobaciones', canMatch: [permisoGuard('gastos:aprobar')], ... },
    { path: 'admin/dashboard',    canMatch: [permisoGuard('dashboard:ver')], ... },
  ],
},
```

Redirección contextual [Hecho]:

- Sin sesión, `authGuard` reconstruye la URL pedida a partir de los *segments* y redirige a `/login?returnUrl=/admin/dashboard`. Tras el login, `LoginPage` vuelve a esa URL.
- Sin permiso, `permisoGuard` manda al usuario a `/inicio?denegado=/admin/dashboard`, y la página de inicio le avisa con un mensaje accesible (`role="alert"`).
- Una sesión caducada (`exp`) se cierra en el siguiente cambio de pantalla (`haySesionValida()`).

Registro del interceptor [Fase 3]. Se registra **una sola vez**, en la configuración global, para que lo atraviesen todas las peticiones de `HttpClient`. Los datos SQL se leerán de la API REST de Supabase (`/rest/v1/...`) con `HttpClient`, precisamente para que pasen por él.

```
// src/app/app.config.ts (Fase 3)
provideHttpClient(withInterceptors([authInterceptor]))

// src/app/core/auth/auth.interceptor.ts (diseño)
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const token = auth.accessToken();                       // del servicio central
  const esApiPropia = req.url.startsWith(environment.supabaseUrl);
  const peticion = token && esApiPropia
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;
  return next(peticion).pipe(
    catchError((err: HttpErrorResponse) => {
      if (err.status === 401) auth.cerrarSesionYVolverAlLogin(); // tras intentar 1 refresh
      if (err.status === 403) auth.avisarSinPermiso();
      return throwError(() => err);
    }),
  );
};
```

B.2 Control visual condicional (UI/UX)

Menú lateral filtrado con una signal computada [Hecho]. El shell recalcula las opciones solo cuando cambian los permisos. Un empleado no ve "Aprobaciones" ni "Cuadro de mando", así que no puede intentar entrar desde el menú.

```
// src/app/layout/shell/shell.component.ts
public readonly menu = computed(() => {
  const permisos = this.auth.permisos();
  return MENU.filter((item) => !item.permiso || permisos.includes(item.permiso));
});
```

- **Cabecera de sesión y cierre [Hecho]**. El menú muestra el nombre, el rol y el departamento. "Cerrar sesión" usa `replaceUrl` para que el botón Atrás no vuelva a la zona privada.
- **Botones de acción** (aprobar, exportar) **[Fase 2-3]**. Se mostrarán con `@if (auth.tienePermiso('gastos:aprobar'))`. Cuando se repita en varias pantallas se encapsulará en una directiva estructural `*appTienePermiso`.
- **Ocultar no es proteger.** La interfaz solo evita errores al usuario; la seguridad la ponen los guards en el cliente y, sobre todo, las **políticas RLS** de Supabase en la base de datos **[Fase 3]**.

C. Matriz de alternativas evaluadas

Guards funcionales vs. comprobación en el componente (`ngOnInit` / `ionViewWillEnter`)

Criterio	Guard en el router (elegido)	Comprobación en el componente
Seguridad	Se decide antes de crear la página: nunca se pinta, ni un instante.	La página se crea y se ve un momento antes de echar al usuario. Fácil de olvidar en una pantalla.
Red / rendimiento	Con `canMatch` no se descarga el código de la página prohibida.	El chunk de la página se descarga siempre.
Mantenibilidad	Toda la política está visible en `app.routes.ts`.	Lógica de acceso repetida y dispersa por los componentes.
Escalabilidad	Una ruta nueva solo necesita añadir su guard.	Cada pantalla nueva tiene que copiar la comprobación.

Interceptor HTTP vs. cabeceras manuales en cada servicio

Criterio	Interceptor (elegido)	Cabeceras manuales
Seguridad	Un único punto controla a qué dominios se envía el token.	Riesgo de olvidar la cabecera o de mandarla a un tercero.
Errores 401/403	Gestión centralizada y renovación compartida.	Cada servicio trata el error a su manera.
Mantenibilidad	Cambiar el esquema de autenticación toca un archivo.	Hay que tocar todas las llamadas.
Coste	Una función más en la cadena (despreciable).	Código duplicado.

RBAC vs. ABAC vs. flag booleano (`isAdmin`)

Criterio	Flag `isAdmin`	RBAC (elegido)	ABAC (por atributos)
Expresividad	Solo dos estados.	Roles con conjuntos de permisos.	Reglas con atributos del usuario, del recurso y del contexto (hora, importe...).
Escalabilidad	Un rol nuevo obliga a cambiar todo el código.	Un rol nuevo (p. ej. `supervisor`) es una línea en la matriz.	Muy flexible, pero las reglas crecen rápido.
Mantenibilidad	Condiciones `if (isAdmin)` repartidas por todo el código.	Matriz única y legible.	Necesita un motor de políticas y es más difícil de auditar.
Encaje en GastoFlow	Insuficiente.	**Adecuado**: dos o tres roles con permisos claros.	Excesivo. Lo único "por atributo" (un empleado solo ve **sus** gastos) se resuelve en la base de datos con RLS (`empleado_id = auth.uid()`).

Conclusión. GastoFlow combina **guards funcionales con `canMatch`** (y sin precarga), un **interceptor HTTP único** y **RBAC con permisos atómicos** centralizado en `AuthService`. Así hay tres capas independientes: la interfaz (qué se muestra), el router (a dónde se puede ir) y la base de datos con RLS (qué datos se pueden leer o modificar). Si falla una, las otras siguen protegiendo los datos.